

ENGINEER ONBOARDING GUIDE

Action Stories

Ye document batata hai poora system kaise kaam karta hai — URL kholne se lekar screen dikhne tak, har step diagram ke saath. Saath hi ye bhi ki "template" kaise ek hi rehte hue dynamically 26 alag workflows (104 screens) bana deta hai, aur naya workflow add karne ke liye tumhe kya karna hoga.

DIAGRAM COLOR KEY (PURA DOCUMENT ISI RANG MEIN HAI)

- ☐ Start / User action
- ☐ React Component (FIXED code — sabke liye same)
- ☐ Logic / Function (FIXED code)
- ☐ JSON Data (VARIABLE — har screen ka apna)
- ☐ Final rendered result

SECTION 1

30-Second Summary

Ye ek **generic renderer** hai. Koi bhi screen ke liye alag React component nahi likha gaya — sirf **1 engine** hai jo **2 JSON files** (ek "structure" file + ek "data" file) padh ke kisi bhi screen ko render kar deta hai. 26 alag "workflows" (S9.1...S9.20, S10.1...S10.6), har ek ke 4 stages (Reason → Analyze → Decide → Execute) — total ~104 screens, sab ek hi engine se banti hain.

Yaad rakhne wali sabse important line: "Code jo screen dikhata hai" hamesha same rehta hai. "Data jo us code ko diya jata hai" har screen ke liye alag hota hai. Isi wajah se ye "template" system hai — jaise ek hi invitation-card ka design (template) fix rahe, aur sirf naam/date/venue (data) badalta rahe.

Agle sections mein hum yehi cheez step-by-step, diagram ke saath dekhenge:

1. **Section 2** — URL kholne se screen dikhne tak ka poora functional flow (diagram)
2. **Section 3** — Template dynamic kaise hai: Fixed Engine vs Variable Content (diagram)
3. **Section 4** — Ek value literally kahan se aati hai — close-up example (diagram)
4. **Section 5** — File/folder map — kaunsi file kya hai aur kyun
5. **Section 6** — Naya workflow/template kaise add karoge (step diagram)
6. **Section 7** — Abhi ka data (fake) vs future real API — kya badlega
7. **Section 8** — Rozana kaam ke recipes + abhi kya adhoora hai

SECTION 2

Poora Functional Flow — URL se Screen tak

Jab koi user `/action-stories/S9.1/reason` khol kar aata hai, ye 9 steps hote hain, is order mein:

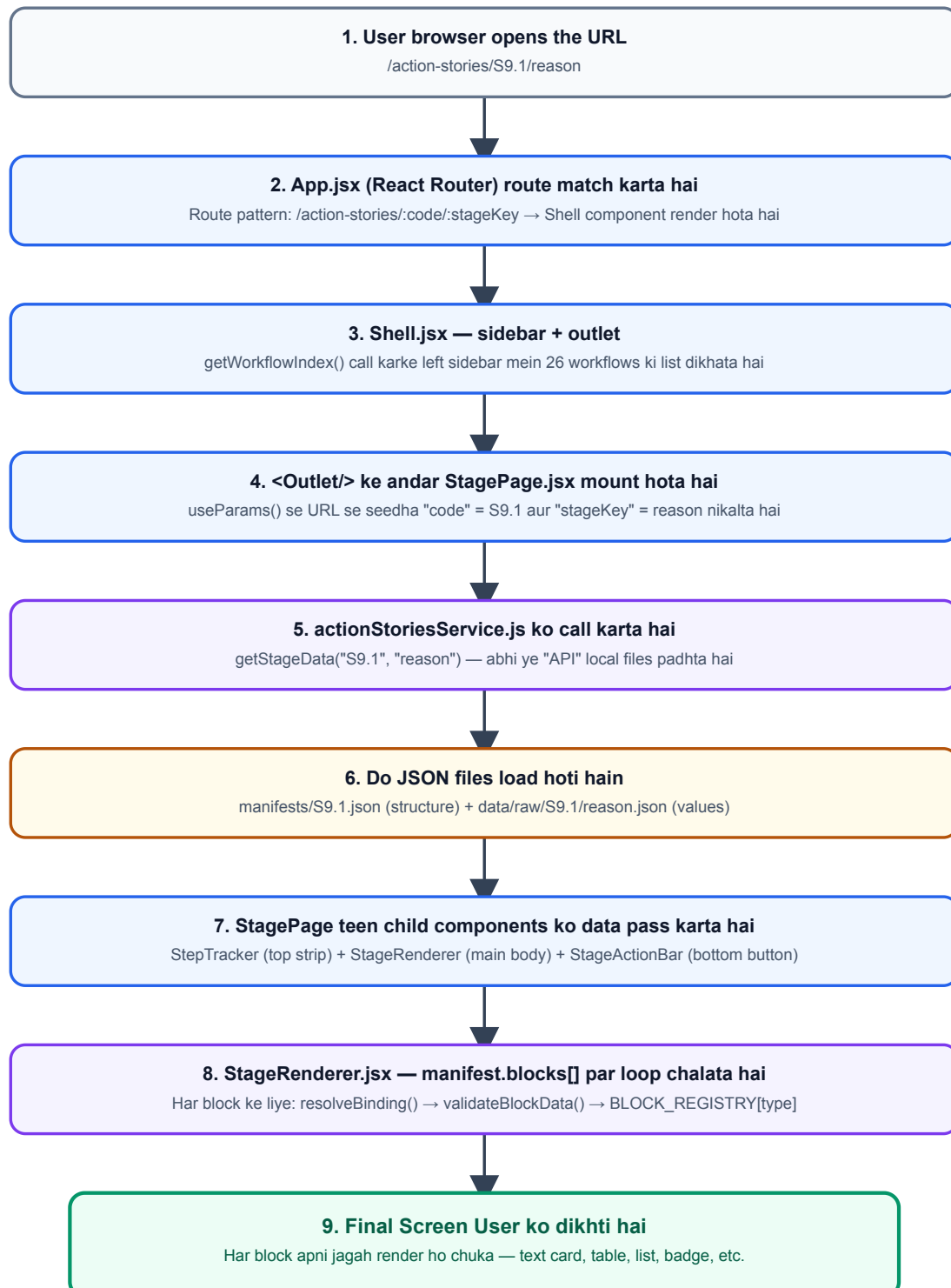


Diagram 1 — Poora request-to-render lifecycle, top se bottom

Crash-proof design: Step 8 ke andar agar koi binding galat hai, blockType unknown hai, ya data wrong shape mein hai — poori screen crash nahi hoti. Sirf wo ek block ek chhota amber "placeholder" box dikhata hai + console mein warning

print hoti hai. Baaki sab blocks normal render hote rehte hain.

Kahan hai "current workflow/stage" ki state? Kahin bhi component state ya store mein save NAHI hai — hamesha URL mein hi hai (`useParams()`). Isliye browser refresh ya back/forward button dabane par bhi sahi screen hi dikhti hai, kabhi stale nahi hoti.

SECTION 3

Template Dynamic Kaise Hai — Fixed Engine vs Variable Content

Jo confusion hoti hai naye logon ko — "104 screens hain to 104 jagah code hoga na?" — **Nahi**. Neeche wala diagram exactly yahi dikhata hai: left side jo bhi hai wo ek hi baar likha gaya hai aur sabke liye reuse hota hai. Right side har screen ke liye alag hai, lekin wo sirf JSON hai — code nahi.

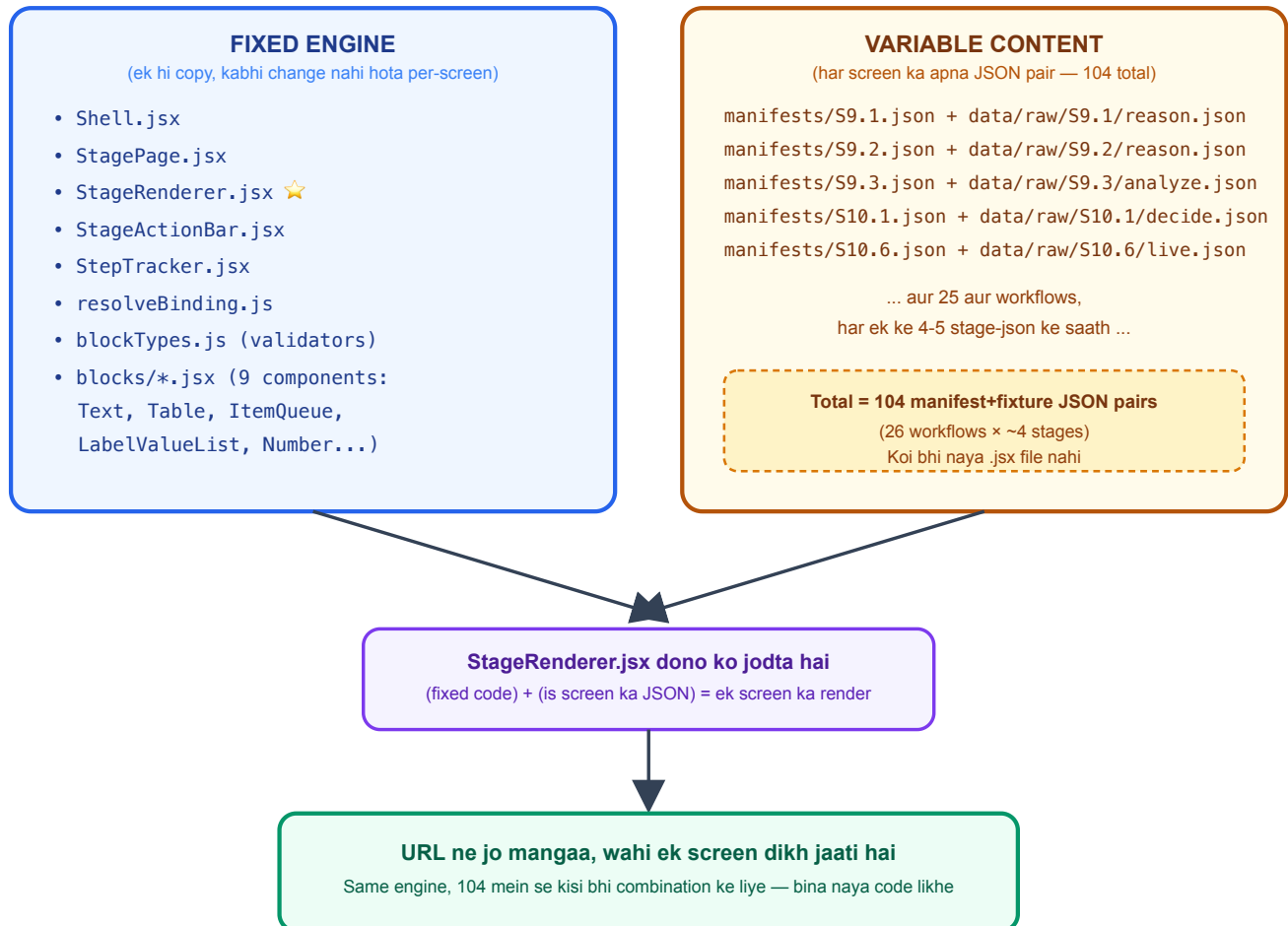


Diagram 2 — "Template" ka asli matlab: 1 engine + बहुत सारी JSON "content" pairs

Isiliye ye "dynamic" hai: Agar kal ko 27 vaan workflow chahiye, koi bhi naya `.jsx` file nahi banani. Sirf ek naya JSON manifest + ek naya JSON fixture daal do — engine automatically use render kar dega. Detail Section 6 mein.

SECTION 4

Ek Value Kahan Se Aati Hai — Close-up Example

Chaliye ek asli example lete hain: `S9.1 / reason` stage ka pehla block, jiska naam "rationale" hai. Dekho iske peeche ka poora path kya hota hai:

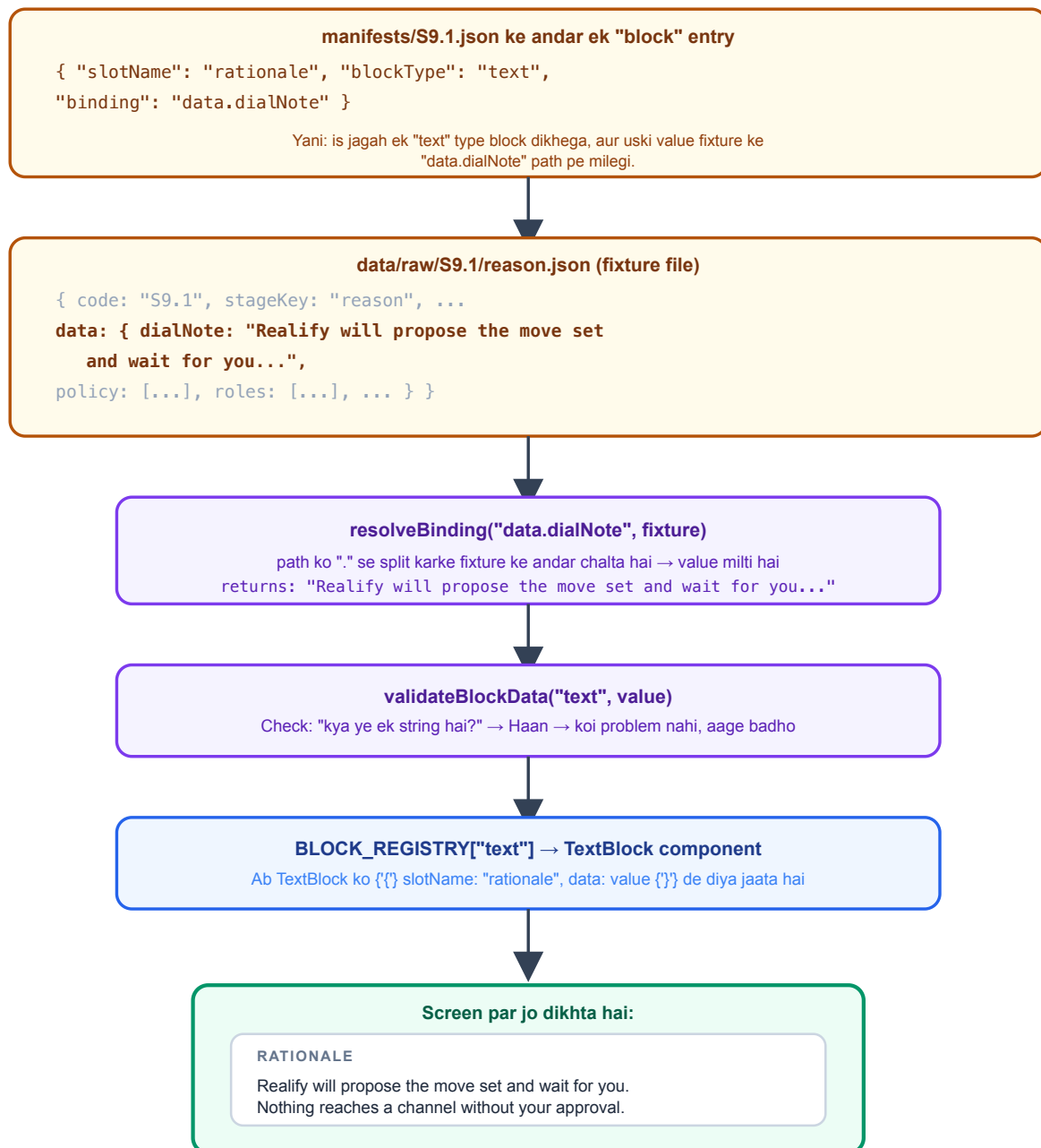


Diagram 3 — Ek single value "binding string" se leke "screen pe dikhna" tak

Ye **exact same 4-step pipeline (resolve → validate → lookup → render)** har ek block ke liye chalti hai — chahe wo text ho, table ho, ya kuch aur. Sirf ye 4 steps yaad rakho aur tum kisi bhi "ye value kahan se aa rahi hai" wale sawaal ko khud trace kar sakte ho.

SECTION 5

File aur Folder Map

Neeche rang-coded tree hai. Blue = fixed engine code, Amber = variable JSON data, Purple = logic/helper functions.

```
src/
├─ App.jsx Router – sab routes yahan
├─ constants/actionStoriesRoutes.js URL banane ke helpers
├─ services/actionStoriesService.js "API layer" – abhi local JSON padhta hai
├─ store/useActionStoriesStore.js Sirf "Approve button dabaya?" yaad rakhta hai
├─ features/action-stories/
│   └─ components/
│       ├── Shell.jsx Sidebar + <Outlet/> (app ka frame)
│       ├── StageRenderer.jsx ★ MAIN ENGINE – manifest ko fixture se jodta hai
│       ├── StageActionBar.jsx Neeche ka Approve/Start button
│       └─ StepTracker.jsx Upar ki Reason→Analyze→Decide→Execute strip
│   └─ pages/
│       ├── ActionStoriesHome.jsx "/" khulte hi pehli screen pe redirect
│       └─ StagePage.jsx URL padhta hai, data fetch karta hai, jodta hai
├─ manifests/
│   ├── S9.1.json ... S10.6.json (×26) "kya dikhega" – STRUCTURE
│   ├── resolveBinding.js binding string → value nikalta hai
│   ├── blockTypes.js 9 block types + unke validators
│   ├── validateManifest.js manifest file khud sahi hai ya nahi
│   └─ REPORT.md real-data field mapping notes (padhna zaroor)
├─ blocks/ 9 "dumb" render-only components
│   (Text, Number, Flag, LabelValueList, Table, ItemQueue, Series, Object, Slider)Block.jsx
├─ data/
│   ├── index.json Sab 26 workflows ki list
│   └─ raw/<CODE>/<stage>.json (×104) "asli data" – VALUES
```

Extra folder — app ka part NAHI (sirf build-time tool)

`extraction/` — designer ki mockup files (`source-mockups/*.dc.html`) se `manifests/*.json` aur `data/raw/**/*.json` generate karne wala one-time script. `npm run extract` se chalta hai. Roz ke development mein isse touch karne ki zarurat nahi — sirf tab jab naye design-mockups aayen.

Naya Workflow/Template Kaise Add Hoga (Dynamic Extension)

Ye hi "kitna andar change hoga" wala jawab hai — agar 27 vaan workflow add karna ho, to:

1

Manifest JSON banao

Naya file

`manifests/S27.json`

— har stage ke liye blocks ki list (slotName, blockType, binding)

2

Fixture JSON banao

Naya folder

`data/raw/S27/` —
reason.json, analyze.json,
decide.json, execute.json
mein asli data

3

Index update karo

`data/index.json`

mein ek entry: `{code,
name, stages}`

4

Done. Zero code change

Engine khud detect karke
render kar dega — koi .jsx
file nahi banani

Naya "block type" chahiye ho (jaise koi chart-type block) to thoda alag hai — ye tab hota hai jab koi *naya shape* chahiye jo maujooda 9 types mein fit nahi hota:

1. `blocks/MyNewBlock.jsx` banao (render-only component)
2. `manifests/blockTypes.js` mein `validateMyNewBlock()` likho
3. `blocks/index.js` mein ek line add karo: `myNewType: MyNewBlock`

`StageRendererer.jsx` ko haath lagane ki zarurat NAHI — wo blindly registry mein lookup karta hai.

Kisi existing screen ka field/data badalna ho to?

Bas do jagah dekho: uska `manifests/<CODE>.json` (kya block dikhega) aur `data/raw/<CODE>/<stage>.json` (kya value dikhegi). Kabhi bhi kisi `.jsx` component ko is kaam ke liye chhedne ki zarurat nahi padegi.

Abhi ka Data (Fake) vs Future Real API

Ye poora project abhi ek "scaffold/prototype" hai — real backend se connect nahi hai. Sara data `actionStoriesService.js` local JSON files se padhta hai. Jab real backend banega, sirf **isi ek file** mein change hoga — baaki poora system (jo humne upar dekha) bilkul waisa hi rahega.

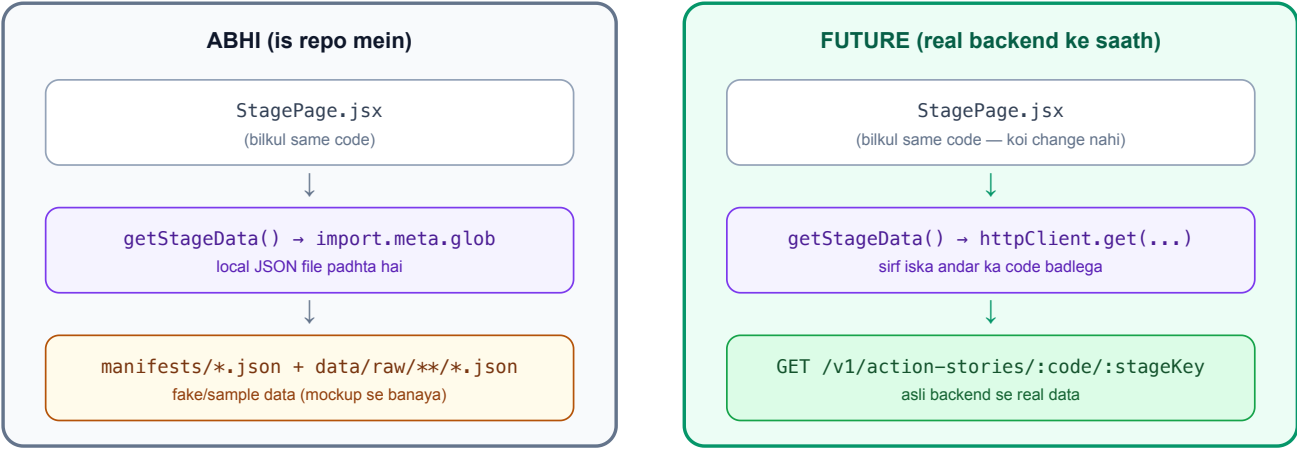


Diagram 4 — Sirf ek file (`actionStoriesService.js`) badlegi, baaki sab wahi rahega

⚠ Field-name gotcha (INTEGRATION.md se, zaroor padhna): Kuch field names jo abhi mockup-data mein hain — jaise `execution_lane` aur `guardrail_verdict` — real backend mein bhi wahi naam se maujood hain, lekin unka **matlab bilkul alag** hai. Real API connect karte waqt bina check kiye seedha wire kar diya to galat cheez screen pe dikh sakti hai. Poora comparison table root folder ki `INTEGRATION.md` file mein hai.

SECTION 8

Rozana Kaam Ke Recipes

Karna kya hai	Kahan jao
Screen pe naya field dikhana hai	<code>manifests/<CODE>.json</code> mein naya <code>{slotName, blockType, binding}</code> add karo + fixture mein wo data ho
Ek block ka look/style badalna hai	<code>blocks/<TypeName>Block.jsx</code> — sabhi 104 screens pe apply hoga (kyunki fixed hai)
Naya workflow/stage add karna hai	Naya <code>manifests/<CODE>.json</code> + <code>data/raw/<CODE>/*.json</code> + <code>data/index.json</code> mein entry
Naya block type chahiye	<code>blocks/NewBlock.jsx</code> → <code>blocks/index.js</code> mein register → <code>blockTypes.js</code> mein validator
Real backend se connect karna hai	Sirf <code>services/actionStoriesService.js</code> badlo (details: <code>INTEGRATION.md</code>)
Approve button ko real save karana hai	<code>store/useActionStoriesStore.js</code> ka <code>TODO(action-stories-mutations)</code> comment dekho
Kisi field ka real API meaning check karna hai	<code>manifests/REPORT.md</code> + <code>INTEGRATION.md</code> ka table

Abhi kya incomplete/decorative hai (confuse mat hona)

- **Approve/Start button** — sirf UI hai, koi backend save nahi hota (refresh karoge to reset ho jayega)
- **Slider block** — koi bhi screen abhi use nahi kar rahi; code future ke liye ready hai
- **recharts library** — installed hai but use nahi ho rahi; `SeriesBlock.jsx` abhi raw SVG path se chart dikhata hai

Golden rule jab bhi confuse ho: "ye value kahan se aa rahi hai" — trace karo hamesha isi order mein: **URL** → **manifests/<CODE>.json (kaunsa block)** → **binding string** → **data/raw/<CODE>/<stage>.json (asli value)**. Yehi 4-step pipeline (Diagram 3) har jagah repeat hoti hai.